

PRÁCTICA 1: REPASO DE JAVA

Rodrigo García Carmona¹
rodrigo.garcia@upm.es



Esta práctica es individual y sirve tanto para desarrollar como para poner a prueba los conocimientos adquiridos. Realizar esta práctica es recomendable para aumentar las probabilidades de éxito en los exámenes teóricos.

1. Preparación de la práctica

El objetivo de esta práctica es refrescar los conocimientos de Java, ya que, en condiciones ideales, hará dos años que superó la asignatura Fundamentos de Programación de primer curso. En esta práctica repasará los siguientes conceptos:

- Uso de tipos de datos primitivos, arrays, objetos y colecciones.
- Uso de sentencias de control de flujo.
- Creación de algoritmos sencillos.
- Lectura y escritura de archivos.
- Implementación de interfaces y estructuración de proyectos *software*.

TRABAJO PREVIO AL LABORATORIO

El tiempo de las clases de laboratorio es muy valioso, ya que en ellas tendrá a su disposición a los profesores para resolver cualquier duda que le pueda surgir. Por tanto, para facilitar al máximo el aprovechamiento de este tiempo, las prácticas se han dividido en varias secciones. La primera de ellas (“preparación de la práctica”) puede ¡y debe! realizarse **antes de la sesión de laboratorio** propiamente dicha. Leer esta sección antes de entrar a clase hará que su tiempo en el laboratorio sea más productivo y provocará que las dudas más importantes le surjan **durante la clase**, y **no después**, cuando se encuentre estudiando solo en su casa y la única opción que le quede sea escribir un email al profesor.

De hecho, es especialmente importante evitar que se den situaciones como esta la noche antes de un examen.

1.1. OpenBCI

Los BCI (*Brain Computer Interfaces*, interfaces cerebro-ordenador) son dispositivos que permiten interactuar con un ordenador a través de las señales eléctricas que produce el cerebro humano, las mismas que forman la base del pensamiento. Esto hace posible, usando un lenguaje coloquial, “controlar un ordenador con la mente”.

Existe una gran variedad de dispositivos BCI, que se pueden clasificar, a grandes rasgos, en invasivos y no invasivos. Los invasivos se introducen directamente en contacto con el cerebro de una persona (o animal), mientras que los no invasivos no precisan de ninguna operación quirúrgica o incisión. Lógicamente, los primeros permiten acceder a las señales eléctricas del cerebro de forma directa y obtienen datos de más calidad, pero tienen la contrapartida de que implican la inserción de un objeto extraño en el interior del cuerpo humano, lo que puede ocasionar rechazos, precisar de operaciones constantes (las neuronas se mueven dentro del cerebro) y otros muchos problemas.

¹Desarrollada a partir del trabajo previo de Miguel Ángel de Miguel.

Hoy en día, cuando el común de los mortales piensa en dispositivos BCI, lo más probable es que lo primero que le venga a la mente (je) sea Neuralink, la empresa del polémico Elon Musk, que está desarrollando dispositivos invasivos. Pero no es la única que investiga en el campo de los BCI. En el otro extremo se encuentra [OpenBCI](#), una empresa pequeña (en comparación con Neuralink) que se financió a través de campañas de *crowdfunding* y que produce dispositivos BCI no invasivos que cualquier persona con los medios necesarios puede construir, pues es un proyecto de código abierto (*open source*) y *hardware* abierto (*open hardware*).

OPEN SOURCE Y OPEN HARDWARE

El *open source* es un movimiento (o una filosofía, según a quién pregunte) que promueve que el código fuente de los programas informáticos sea hecho público (“liberado”). Este movimiento afirma que, si uno solo dispone de los programas compilados, sin el código fuente, no es verdadero dueño del *software* que utiliza, ya que no puede modificarlo, mejorarlo o utilizarlo de formas distintas a las que concibió quien se lo vendió. Además, no disponer de este código impide saber qué está haciendo realmente el *software* y nos hace dependientes del fabricante original para obtener soporte o reparaciones. El movimiento *open source* está muy relacionado con otros como el *right to repair* (derecho a reparar), el *maker manifesto* (democratizar la fabricación de dispositivos tecnológicos), la cultura *hacker*, el *open access* (ciencia abierta) o el *open hardware* (*hardware* que viene acompañado de las instrucciones para replicarlo), entre otros.

Si siente curiosidad, investigue estas u otras iniciativas relacionadas. Quizá entregar el acceso y control sobre nuestros pensamientos a una empresa privada que ejecuta *software* cuyo funcionamiento real desconocemos no sea la mejor de las ideas.

Hasta la fecha, OpenBCI ha creado dos placas, la OpenBCI 32 Bit (ver Figura 1) y la Ganglion, que cuentan con amplificadores biométricos y que se pueden conectar a electrodos que se colocan sobre la piel para capturar varias señales eléctricas producidas por el cuerpo humano: EMG (electromiograma), ECG (electrocardiograma) y EEG (electroencefalograma).

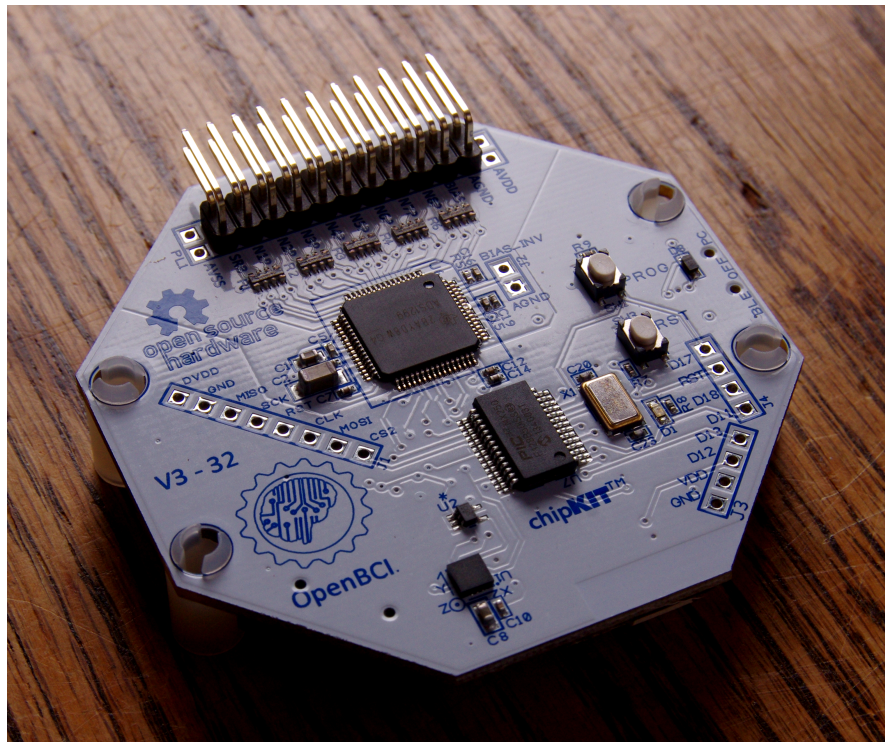


Figura 1: Placa OpenBCI 32 Bit²

²[Omphalosskeptic - CC BY-SA 4.0](#)

Estas señales, una vez capturadas, pueden transmitirse a otros dispositivos a través de protocolos de comunicaciones como Bluetooth, lo que permite integrar los dispositivos que siguen el estándar OpenBCI con, por ejemplo, un *headset* de realidad virtual (ver Figura 2), lo que nos permitiría utilizar estas señales como comandos con los que controlar a un avatar en un entorno virtual.



Figura 2: OpenBCI con el dispositivo de realidad virtual Varjo XR3³

Si dispone de conocimientos de programación y de electrónica, sabe soldar y tiene ganas de cacharrerar, la creación de un entorno como este está a su alcance.

1.2. Grabaciones de EEG

Para obtener las señales eléctricas del cerebro (EEG) de forma no invasiva se utilizan una serie de electrodos colocados en diferentes partes de la cabeza de una persona, habitualmente dispuestos de una forma concreta gracias a un “casco” o “gorro” como el que aparece en la Figura 3.

Cada uno de estos electrodos recibe el nombre de “canal”, y permite capturar las señales eléctricas de una región del cerebro; la más próxima al electrodo. Normalmente, con mejor calidad cuanto menos pelo tenga la persona. Estas señales eléctricas son de una amplitud muy pequeña, del orden de los microvoltios.

OpenBCI registra, a una frecuencia determinada (por ejemplo, 250 Hz), la amplitud obtenida de cada uno de estos canales, y se la entrega al dispositivo al que esté conectado. Esta información también puede registrarse en un archivo, para poder acceder a ella más adelante, analizarla y filtrarla.

Estos archivos son de un tipo llamado CSV (*Comma Separated Value*, valores separados por comas), en los que cada línea representa el EEG medido en un instante de tiempo. Cada línea empieza con un número, que indica el índice de la muestra concreta módulo 256, seguido de tantos *samples* (medidas individuales) como canales haya. Todos los valores de una línea están, como el nombre del archivo sugiere, separados por comas.

¿MÓDULO 256?

Si no sabe lo que significa “módulo 256”, invéstiguelo. Es una operación matemática muy habitual a la hora de desarrollar algoritmos.

³Carillons - CC BY-SA 4.0

⁴Conorrussomanno - CC BY-SA 4.0

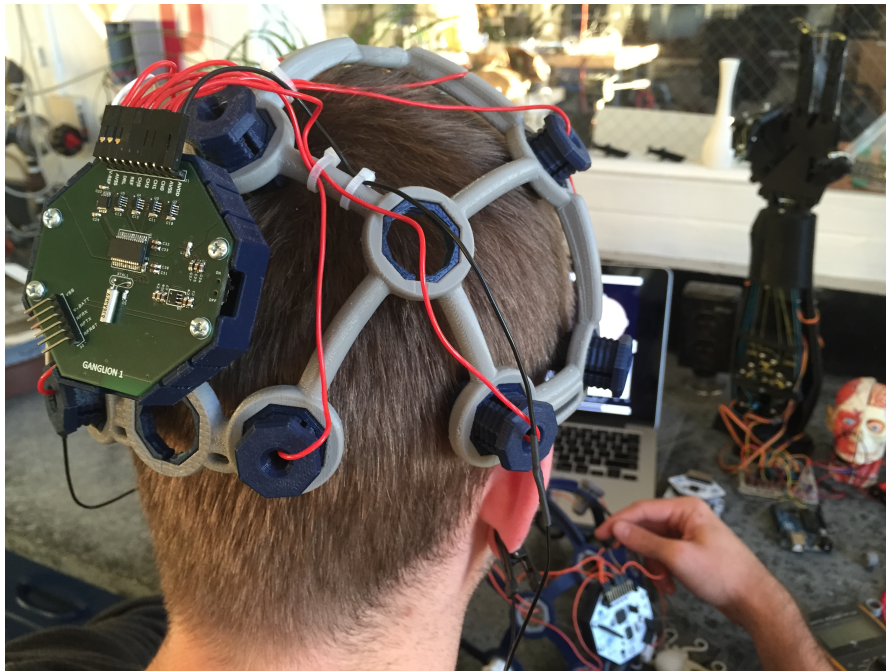


Figura 3: Casco impreso en 3D Ultracortex Mark III⁴

A continuación tiene un archivo de una (sumamente breve) sesión de EEG creado con OpenBCI. En este caso hay 11 muestras de 4 canales (electrodos):

```
0, 214.0, -8.0, 57.0, 70.0
1, 131.0, -2.0, 60.0, 73.0
2, 79.0, 16.0, 83.0, 51.0
3, -47.0, 16.0, 55.0, 52.0
4, 83.0, -1.0, 69.0, 70.0
5, 75.0, 9.0, 69.0, 65.0
6, 168.0, 16.0, 66.0, 64.0
7, 396.0, 8.0, 69.0, 56.0
8, -53.0, -6.0, 59.0, 68.0
9, 166.0, 13.0, 57.0, 87.0
10, 228.0, 7.0, 65.0, 61.0
```

FORMATO DE LOS DATOS

En esta práctica tendrá que procesar y crear ficheros con un tipo de formato (OpenBCI) y para unos datos (EEG) concretos. Este tipo de trabajo es muy habitual en la ingeniería biomédica. De hecho, todos los dispositivos biomédicos deben, de una forma u otra, procesar datos, valga la redundancia, biomédicos.

1.3. Objetivo y descarga de la práctica

El objetivo de la práctica es terminar de desarrollar una aplicación Java que permite visualizar, en forma de gráficas, la información registrada en archivos en formato OpenBCI de grabaciones de EEG. Además, esta aplicación también permite la generación de archivos OpenBCI sintéticos (es decir, no capturados a partir de un sujeto humano, sino generados artificialmente). Este tipo de datos sintéticos pueden ser útiles para probar módulos *software*, equipos o entrenar modelos.

La práctica está disponible en GitHub, en la siguiente dirección de Internet (URI):

<https://github.com/rgarciacarmona/ALED-lab1>

Para trabajar en ella, siga estos pasos:

1. Haga un *fork* de este repositorio a su cuenta de GitHub.
2. Antes de empezar a hacer *commits*, compruebe en *Settings* → *General* de su *fork* que la visibilidad es **pública**. Solo se evaluarán los *commits* hechos sobre un *fork* público.
3. Clone su *fork* e impórtelo en Eclipse. Para ello, seleccione *File* → *Import...* y, de entre las opciones disponibles, *Git* → *Projects from Git* → *Clone URI*. A continuación, introduzca la URI de su *fork* (<https://github.com/usuario/ALED-lab1>, donde *usuario* es su nombre de usuario de GitHub), no la del repositorio original. ¡Ojo, la captura de pantalla que verá más adelante muestra la URL del repositorio original, no la suya (lógicamente)! Después, haga click en *Next* tantas veces como sea necesario y, por último, en *Finish*.
4. Trabaje, haga *commit* de sus cambios y haga *push* a su *fork*.

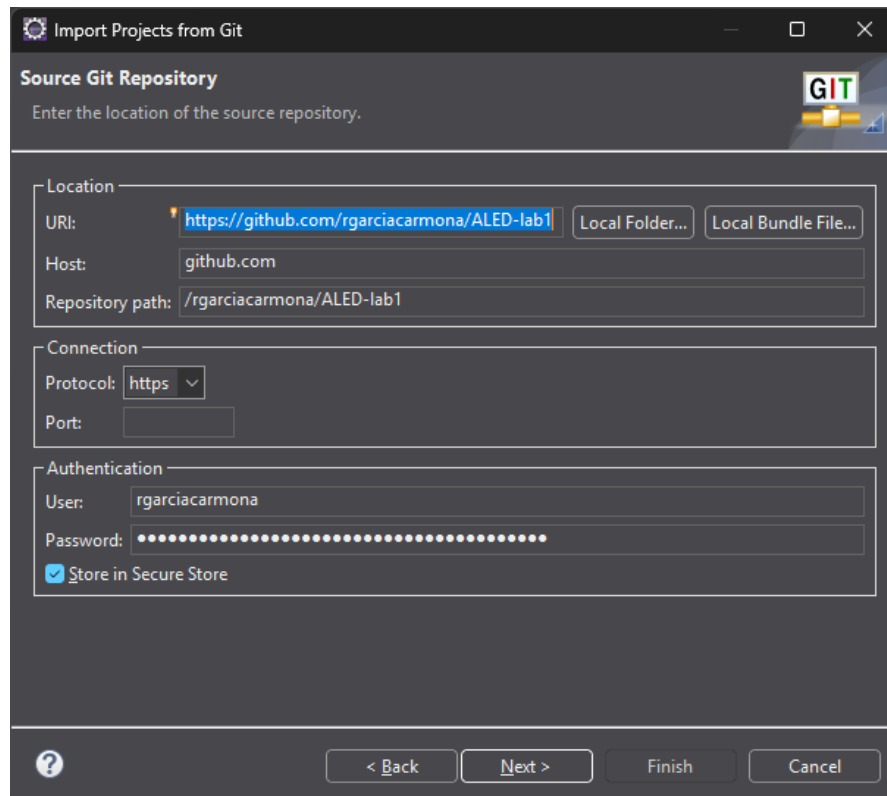


Figura 4: Importar proyecto Git a Eclipse

A continuación, tómese su tiempo para examinar todas las clases del proyecto que acaba de importar. Intente figurarse, a base de observar el código, para qué sirve cada una de ellas. No deje de leer los comentarios; están ahí por algo.

1.4. Modelado de datos

Para que los datos de una sesión de EGG puedan ser manejados por un programa informático, será necesario crear un modelo de los mismos que capture toda la información relevante. Como estamos trabajando en Java, un lenguaje orientado a objetos, se han definido las siguientes clases, que deberían capturar toda la información importante presente en un archivo en formato OpenBCI: `EEGModel` y `Measurement`. Puede ver un diagrama de clases UML de las mismas en la Figura 5 (no se muestran los métodos privados).

Intente interpretar la figura y entender cómo la información contenida en un fichero puede acabar transformada en los objetos que instancian estas dos clases. Después, abra el código fuente de ambas clases (se encuentran en el paquete `es.upm.aled.lab1.measurements`) y confirme que las implementaciones de los métodos hacen lo que usted sospechaba que iban a hacer tras ver el diagrama de clases.

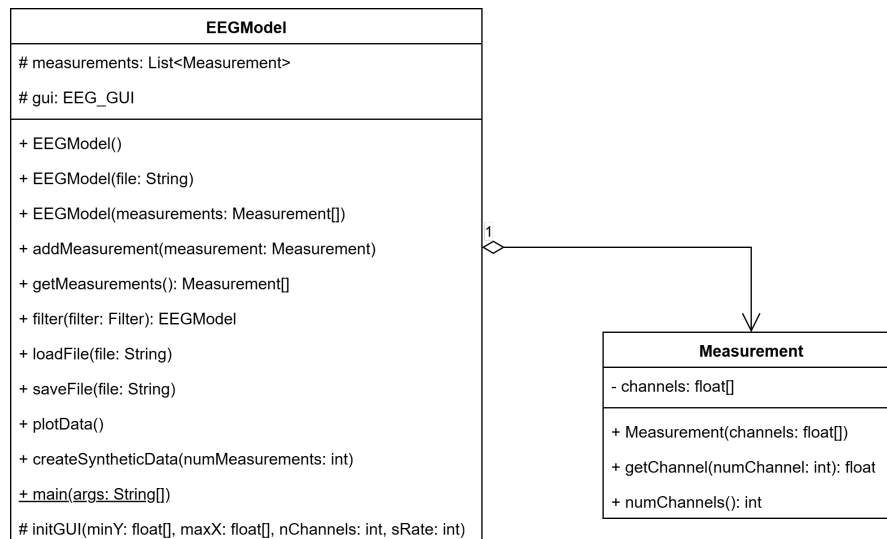


Figura 5: Diagrama de clases del modelo de datos de la práctica

// TODO

Al examinar la clase `EEGModel`, habrá observado que hay una serie de métodos que contienen el comentario “TODO”. Esto sirve para indicar aquellas partes del código que están pendientes de hacer (del inglés “to do”). Estas partes son, como ya habrá imaginado, las que tiene que completar para poder terminar la práctica con éxito.

Consejo: en Eclipse, se puede ver todos los *TODOs* pendientes de un proyecto seleccionando la vista *Window* → *Show View* → *Tasks*.

Responda a las siguientes preguntas:

- ¿Qué significan los símbolos “+”, “-” y “#” en el diagrama de clases?
- ¿Qué significa la flecha que conecta `EEGModel` con `Measurement` en el diagrama?
- Fíjese en el archivo de ejemplo de una grabación de EEG en el formato OpenBCI que aparecía en la sección 1.2. Piense en qué objetos concretos de las clases `EEGModel` y `Measurement` podrían representar este fichero. ¿De qué tamaño será la `List measurements` del objeto de la clase `EEGModel`? ¿De qué tamaño será el array `channels` de los objetos de la clase `Measurement`?
- ¿Cuántos métodos tiene `EEGModel`? ¿Cuáles aún no están implementados?
- ¿Cuántos argumentos puede aceptar el método `main` de la clase `EEGModel`? ¿Cuántas formas de funcionamiento tiene este método y qué hace cada una de ellas?

Enhorabuena, ya ha terminado con el trabajo previo a la sesión de laboratorio. Las secciones de la 2 en adelante puede realizarlas en clase.

1.5. Entrega de la práctica

No es necesario entregar esta práctica ni ninguna de las siguientes. Dado que no se cuentan para la nota, ¿qué sentido tendría hacerlo? Como ya se explicó en la práctica 0, la única razón para hacer las prácticas es aprender.

La única persona que tiene que saber si ha hecho la práctica o no es usted.

2. Guardado en archivos

A continuación va a acabar de desarrollar las funcionalidades que le faltan a la aplicación para estar completa. Lo primero que hará será implementar el método `saveFile` de la clase `EEGModel`,

que, como puede observar en los comentarios, debe guardar en un fichero un `EEGModel`; o, lo que es lo mismo, una grabación de una sesión de EEG.

En primer lugar, fíjese en la carpeta *recordings*, que contiene dos archivos `.txt`. Ábralos y observará que ambos son archivos en el formato OpenBCI que se describió en la sección 1.2. Estos archivos son dos sesiones de EEG guardadas en el formato OpenBCI, el que aplicación deberá leer y escribir.

Responda a las siguientes preguntas:

- ¿Cuántos canales tiene cada uno de los archivos?
- ¿Cuántas muestras tiene cada uno de los archivos?

PAQUETES

El código de la práctica se encuentra dividido en dos paquetes: uno de ellos, `es.upm.aled.lab1.measurements`, contiene las clases relativas al modelado y manipulado de las sesiones de EEG. El otro, `es.upm.aled.lab1.gui`, agrupa las clases necesarias para mostrar visualmente las gráficas de las sesiones. Este tipo de separación en paquetes es habitual en proyectos *software* de cierta complejidad, y es muy útil para que nuestro cerebro no se sature con demasiada información, ocultando y apartando aquella que no es necesaria en un momento dado. En ingeniería del *software*, a esto se le llama “separación de intereses”.

La buena noticia de todo esto es que, para realizar esta práctica, solo tendrá que fijarse en las clases del paquete `es.upm.aled.lab1.measurements`. El resto, en lo que a usted respecta, “simplemente funcionan”.

2.1. Ejecutar el método `main`

Como ya pudo observar, el método `main` tiene dos formas de funcionamiento:

1. Recupera las muestras de una sesión de EEG almacenada en un archivo.
2. Crea una nueva sesión de EEG sintética con muestras aleatorias.

Cuál de las dos formas de funcionamiento se ejecuta dependerá del número de argumentos que se le pasen al `main`. Por defecto, al ejecutar el método `main` de una clase, Eclipse no le pasa ningún argumento. Por tanto, si hacemos una ejecución directa lo que hará será crear una sesión de EEG sintética.

Para pasarle parámetros al método `main` desde Eclipse tendremos que crear una configuración de lanzamiento. Para ello hay que hacer click-derecho en la clase que contiene el `main` (en nuestro caso, `EEGModel`), y seleccionar *Run As* → *Run Configurations...* Esto nos llevará a una pantalla en la que podremos especificar argumentos (opción *Program arguments*) e indicar desde qué carpeta va a ejecutarse la aplicación (opción *Working directory*).

En la Figura 6 puede ver cómo configurar estos parámetros para que el método `main` reciba como argumento el nombre de uno de los dos ficheros y se ejecute en la carpeta *recordings*.

Queremos que la aplicación se ejecute en la carpeta *recordings* por comodidad: cada vez que lee o escribe un fichero, la aplicación espera encontrarlo en la carpeta en la que se está ejecutando. De no ser así, hay que especificárselo manualmente (poniendo, por ejemplo *recordings/OpenBCI_raw_1.txt* en vez de simplemente *OpenBCI_raw_1.txt*).

A continuación, ejecute la aplicación pasándole como argumento cualquiera de los dos archivos que se encuentran en la carpeta *recordings*. Tenga en cuenta que algunos canales no cambian, y que otros pueden tardar más de un minuto en experimentar cambios apreciables. Después, ejecute la aplicación de tal forma que se genere una sesión de EEG sintética.

Responda a las siguientes preguntas:

- ¿Cuántos canales tiene la sesión sintética? ¿En qué parte del código se controla este valor?
- ¿Cuántas muestras tiene la sesión sintética? ¿En qué parte del código se controla este valor?

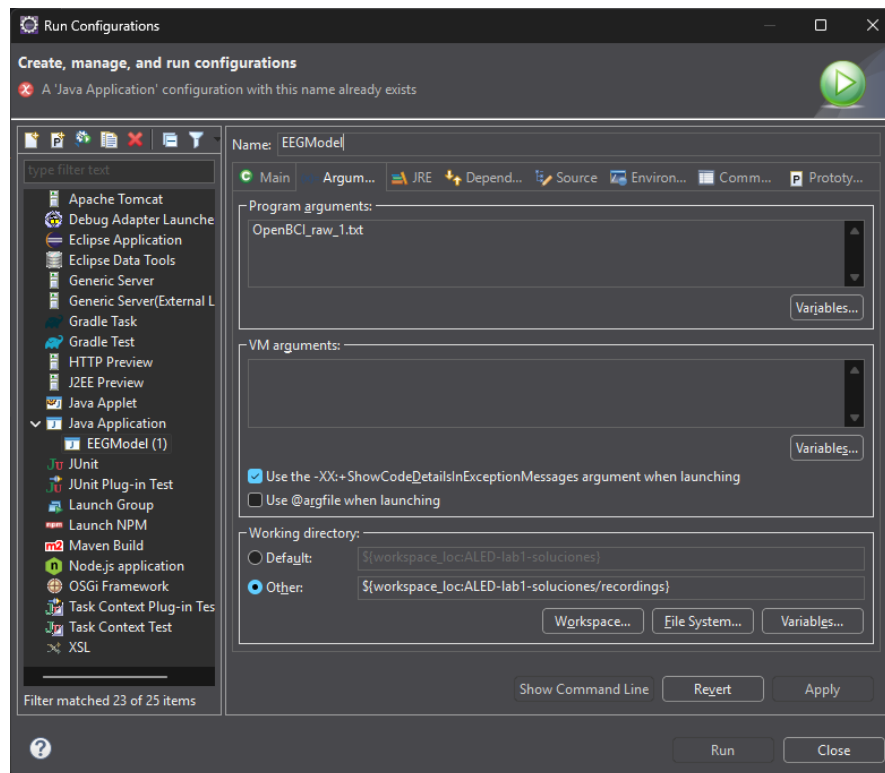


Figura 6: Configuración de lanzamiento de ejemplo

2.2. Implementar el guardado en archivos

Ahora debe implementar la funcionalidad necesaria para guardar una sesión (un `EEGModel`) en un archivo de texto siguiendo el formato CSV de OpenBCI. Es decir, el mismo formato que el método `loadFile` de la clase `EEGModel` espera encontrar. Esta funcionalidad estará contenida en el método `saveFile` de la clase `EEGModel`, por lo que deberá implementarlo.

Para ello, puede utilizar las clases `File`, `FileOutputStream` y `PrintStream` del paquete `java.io`. Recuerde que:

- `File` le permite crear un objeto que represente un archivo a partir de su ubicación.
- `FileOutputStream` le permite crear un *stream* de salida hacia un archivo.
- `PrintStream` le permite escribir cadenas de caracteres a un *stream* de salida mediante los métodos `print` o `println`. Al terminar de usar un *stream*, no olvide cerrarlo. Si no, la información no quedará guardada en el archivo.

System.out

Sí, `System.out` es un objeto de la clase `PrintStream`. Por eso se puede llamar a los métodos `print` y `println` sobre él.

Después, modifique el método `main` para que, al terminar de generar una sesión sintética la guarde en un archivo `Synthetic.txt`.

Tenga en cuenta que Eclipse tiene la mala costumbre de no actualizar automáticamente la vista del explorador de proyecto cuando se crean archivos nuevos, por lo que es posible que no vea el archivo que acaba de guardar. Para forzar este refresco en una carpeta, haga click-derecho en ella y seleccione la opción *Refresh*.

3. Filtrado de muestras

Muchas veces los análisis de EEG se centran en partes específicas del cerebro. Las variaciones de algunos canales (la actividad de esa zona del cerebro) no son significativas y pueden ser

ignoradas. Y, en otros casos, solo algunos momentos (intervalos de muestras concretos) del EEG son interesantes para el estudio que se está llevando a cabo. Por ejemplo, podríamos analizar las variaciones del EEG cuando un paciente está hiperventilando y observar solo unos canales concretos.

De hecho, al visualizar el contenido de los dos archivos de la carpeta *recordings* probablemente se haya percatado usted mismo de este fenómeno; hay mucha información “inútil”. Además, la obtención de datos en las señales biomédicas es un proceso complejo, ya que la ubicación de los sensores en el cuerpo humano es un proceso difícil y las magnitudes a medir son muy pequeñas.

Para solucionar este problema, va a implementar dos filtros que permitan extraer los valores que más nos interesen de una grabación.

La aplicación proporciona un interfaz llamado **Filter** (ver Figura 7), que representa los métodos que cualquier filtro que deseemos crear debe implementar. En este caso estamos hablando de un único método: **applyFilter**. Este método acepta como argumento un **EEGModel** y devuelve otro **EEGModel** sobre el cual se ha aplicado el filtro en cuestión.

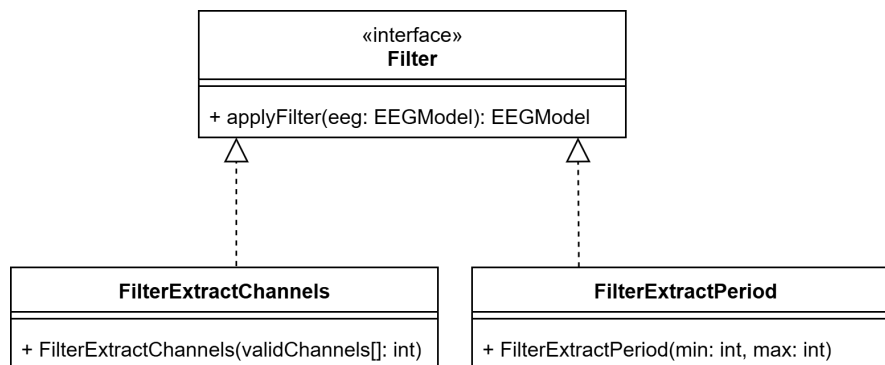


Figura 7: Diagrama de clases de los filtros

Deberá programar dos tipos de filtros diferentes, mediante dos clases que implementan el interfaz **Filter**:

- **FilterExtractChannels:** este filtro genera un nuevo **EEGModel** que solo incluye los canales que se pasan como argumento a su constructor. Así, si el array **validChannels** que se pasa como argumento contiene, por ejemplo, los enteros 3, 6 y 8, el filtro creará un nuevo **EEGModel** con tres canales, que corresponderán a los números 3, 6 y 8 del **EEGModel** sobre el que se ejecuta el método **applyFilter**. En este ejemplo, los tres canales del nuevo **EEGModel** serán los números 0, 1 y 2; y se corresponderán con los canales 3, 6 y 8, respectivamente, del **EEGModel** original.
- **FilterExtractPeriod:** este filtro extraerá un intervalo de muestras concreto, que viene delimitado por los dos enteros (**min** y **max**) que se pasan como parámetros al constructor: **min** será el índice de la primera muestra seleccionada y **max** el de la última muestra seleccionada. Los dos límites deben estar incluidos en el rango. Si, por ejemplo, los argumentos **min** y **max** son 501 y 750, el filtro creará un nuevo **EEGModel** con estas 250 muestras del **EEGModel** original.

Para implementar ambos filtros probablemente necesite implementar también el constructor de **EEGModel** que acepta como argumento un array de **Measurements**. Así que no olvide hacerlo.

Por último, implemente el método **filter** de **EEGModel** que permite aplicar un filtro sobre sí mismo.

Para comprobar que el código que ha programado funciona, modifique el método **main** para que lea el contenido del archivo *OpenBCI_raw_1.txt* y, usando el método **filter**, extraiga solo los 3 últimos canales y las muestras desde la 2750 y 5750, y las dibuje en pantalla. Observará cómo ahora sí que estamos visualizando información relevante.

Enhorabuena, ha terminado la práctica.

4. Extras

Si siente curiosidad, pruebe a llevar a cabo las tareas siguientes:

- Dibuje un diagrama de clases completo del proyecto.
- Investigue cómo funcionan las clases del paquete `es.upm.aled.lab1.gui`.
- Cree un tercer filtro, que tenga dos parámetros en su constructor: un canal (`int`) y un umbral (`float`). Este filtro deberá encontrar la primera muestra cuya medida (*sample*) en el canal especificado supere el umbral, y creará un nuevo `EEGModel` que contenga solo las 100 muestras anteriores, la muestra encontrada, y las 100 muestras siguientes.